ULTIMU PRIZENTI STUDIOS

O projecto por dentro

Código, conteúdo e publicação sob o teu controlo.

1. Contexto e objectivo

Este é o website da empresa de fotografia e produção audiovisual Ultimu Prizenti Studios. A direcção criativa, a identidade visual e o portefólio fotográfico e audiovisual pertencem ao proprietário. A partilha do código no GitHub apresenta um projecto empresarial real e as competências de TI aplicadas ao seu desenvolvimento.

A experiência visual aprovada foi preservada: fundo escuro, tipografia editorial, vídeo de apresentação, grupos fotográficos e páginas individuais. Os três idiomas fazem parte da estrutura desde o início, não são um tradutor sobreposto à página.

2. Arquitectura e escolhas técnicas

O projecto usa JavaScript moderno com módulos ES, HTML e CSS. O Node.js constrói ficheiros estáticos. Não há dependências npm de execução nem um servidor Node em produção: a Netlify distribui os ficheiros de dist. Isto reduz a manutenção e permite mudar de alojamento, embora os formulários precisem de adaptação fora da Netlify.

A organização separa responsabilidades: src/data contém conteúdo; src/pages gera a estrutura específica de cada página; src/components reúne elementos partilhados; public contém os recursos copiados para o resultado final.

3. Como uma página é construída

O ponto de entrada é scripts/build.mjs. A função build lê o catálogo de fotografias, recria dist e copia public. Percorre EN, PL e PT, chama a função de cada página e aplica layout. Depois constrói as colecções e os filmes individuais, a página 404, robots.txt e, quando conhece o endereço público, sitemap.xml.

Exemplo: homePage recebe o idioma, os textos traduzidos e as fotografias. A selecção editorial em projects.mjs define os identificadores e a ordem. photoCard transforma cada fotografia em HTML; layout acrescenta cabeçalho, metadados e rodapé.

As funções devolvem HTML, não alteram uma base de dados. A função escapeHtml protege a inserção de texto em atributos e conteúdo HTML. Os textos do catálogo são dados controlados pelo proprietário; os formulários enviados pelos visitantes não são inseridos automaticamente nas páginas.

4. Interacções no navegador

public/assets/main.js controla o menu móvel, os filtros e a ampliação das imagens, o vídeo de apresentação e os formulários. O elemento dialog fornece a base da galeria ampliada; os controlos permitem navegar pelas fotografias visíveis.

O hero respeita a preferência por movimento reduzido e, quando disponível, a poupança de dados. Os restantes vídeos usam capas reais e só começam quando o visitante escolhe reproduzi-los. Os estilos responsivos mantêm a proporção das fotografias e adaptam grelhas, menu e textos a ecrãs pequenos.

5. Conteúdo multilingue

translations.mjs contém as mensagens de interface; story.mjs guarda a história aprovada; site.mjs reúne contactos e pacotes; projects.mjs reúne filmes, colecções e fotografias seleccionadas; photos.json descreve os originais e as dimensões.

Os endereços começam por /en/, /pl/ ou /pt/. A troca de idioma conserva a página ou projecto. Os segmentos internos permanecem em inglês. SITE_URL ou URL permitem gerar referências absolutas, canonical, alternates e sitemap durante a publicação.

6. Formulários e moderação

Os formulários contact e testimonials são HTML estático detectável pela Netlify. O envio usa POST codificado, com campo form-name e um honeypot anti-spam. O módulo contact-submit.js centraliza o pedido e recusa envios na prévia localhost.

A interface bloqueia cliques repetidos durante o pedido, só confirma após resposta bem-sucedida e conserva os campos se ocorrer um erro. Um timeout não garante que o servidor não recebeu o pedido: por isso a mensagem diz que não foi possível confirmar o envio. Os contactos por email e WhatsApp continuam acessíveis.

As avaliações exigem autorização de publicação. Ficam privadas no painel Forms, com notificações por email depois de configuradas. O proprietário verifica a identidade e o texto; os testemunhos aprovados são acrescentados ao código e publicados novamente. Não existe um painel de aprovação personalizado.

7. Qualidade, segurança e limites

Os testes usam node:test e node:assert. Verificam o número de páginas, os recursos referenciados, as traduções, contactos e preços, a ordem editorial e o contrato dos formulários. A acção do GitHub executa npm run check em cada alteração a main e nos pedidos de revisão. A formatação segue .prettierrc.json e .editorconfig.

public/_headers define políticas de segurança para a Netlify, incluindo CSP e restrições de acesso a câmara, microfone e localização. O servidor local restringe os caminhos a dist e suporta pedidos Range para avançar nos vídeos.

Não existem analytics, pagamentos, contas de clientes ou entrega privada de galerias nesta versão. As promessas comerciais de entrega devem ser cumpridas por um processo externo acordado com o cliente. A privacidade deve ser revista pelo proprietário antes do lançamento, de acordo com a sua actividade real.

8. Como estudar o código

1. Ler package.json para identificar os comandos.

2. Abrir scripts/build.mjs para seguir a compilação.

3. Comparar src/data/projects.mjs com src/pages/home.mjs.

4. Ler src/components/shared.mjs e layout.mjs para perceber a reutilização.

5. Estudar styles.css juntamente com a página aberta em ecrãs diferentes.

6. Seguir um envio desde contact.mjs até contact-submit.js e os testes.

7. Fazer uma alteração pequena, executar npm run check e comparar o resultado.

Comentários explicam intenções e decisões. Evita editar dist: é um resultado gerado, substituído em cada build. Faz commits pequenos e descritivos para manter um histórico útil e conseguir recuperar versões anteriores.

9. Competências demonstradas e evolução

O projecto demonstra decomposição de requisitos, modelação de conteúdo, componentes reutilizáveis, programação JavaScript, CSS responsivo, internacionalização, tratamento de multimédia, testes automatizados, controlo de versões e publicação contínua.

Possíveis evoluções incluem gestão de conteúdos, galerias privadas, streaming adaptativo e um painel de moderação. Não estão implementadas nem são necessárias para apresentar o portefólio actual. Qualquer evolução deve respeitar o orçamento, a experiência aprovada e a independência dos ficheiros.